```
import json
from hotqueue import HotQueue
queue = HotQueue("myqueue", host="localhost", port=6379,
                 db=0)

while True:
      message = queue.get()
      message = json.loads(message)
      '''
      send email to message['recipient'] with
      message['subject'] as subject and
      message['body'] as message body.
      '''
```

In the above example, we created a class by the name *Email*, which is the job class. It has one attribute called *queue*, which specifies the name of the queue in which our messages will be put. Lastly, it implements a static method called *perform*. This method gets the task/message as the only argument, and does what we want our worker to do. So, we are not going to write a separate worker script in case of *PyRes*. Our *Flask* view is pretty much the same as in our previous example, except it uses the *enqueue* method of the *ResQ* class to put tasks in the queue. The first argument is the job class, i.e., *Email*, and the second argument is a JSON encoded message.

```
./pyres_worker email
```

Since we do not have a separate script for our workers, how do we fire them? After installing *PyRes*, you get a *pyres_worker* executable script. So to run your workers, all you have to do is pass this script a comma-separated list of queues, which this worker will listen to. In the above example, we are binding our worker to the *email* queue.

So, *PyRes* is not difficult to implement, and has a lot of advantages. A good reason for you to use it is that GitHub has used *Resque* in production to do millions of task per day. Read the case study on *Resque at https://github.com/blog/542-introducing-resque* to get a better idea.

Message Queues

So far, we have seen some ways of implementing queues using a program or script that handles the queue logic, and makes use of an existing data store to store messages. Let's take a closer look at Message Queues (MQs). They are dedicated systems for the purpose of queuing to enable asynchronous processing of tasks. In simple terms, MQs are systems that can be your queues, and take care of themselves. So instead of a data store, you have the MQ solution or a *Message Queue Broker,* and task-management scripts, which add tasks and take them out to pass them to your workers.

MQs can help you in all the use-cases we discussed earlier and in every scenario where queues are used. The biggest advantage of using MQs is that they are reliable. And that is something that can be important for your application, depending on how important the tasks you want to queue are.

Another great use of MQs (more specifically, with a task-management system called *Celery)* is that you can use it to replace *cron* jobs. I have never been able to use *cron* jobs very smoothly. Using *Celery* for these will bring all your scheduled and repeated tasks into your code-base, and it will be a lot easier and more intuitive to create them and maintain them. When your server crashes, you will not have to take care of your *crontab* separately. Just do a *git clone* and have your *Celery* workers and your Message Queue Broker running, and you are good to go!

RabbitMQ + Celery

RabbitMQ is an MQ solution or a Message Queue Broker very common in the Python community. *Celery*, as the project website states, is an asynchronous task queue or job queue based on distributed message passing. In simple words, *Celery* is a queue or a task manager. The good thing about *Celery* is that it has lots of exciting features, amazing documentation, awesome community support and it not only works with *RabbitMQ*, but with a range of data stores like Redis, MongoDB, and RDBMSs like MySQL. To get started, you must have *RabbitMQ* installed. On Ubuntu/Debian, run *apt-get install rabbitmq-server;* on Fedora, run *yum install rabbitmq-server;* after which you can install Celery with *pip install celery* and you're done.

To define your tasks, *Celery* provides you with some really simple APIs like the task decorator:

```
# file name -> tasks.py
# module name -> tasks

from celery import Celery

celery = Celery('tasks', broker='amqp://guest@localhost//')

@celery.task
def email(recipient, subject, message):
    '''
        send email using whichever library you like
```

We have just defined our tasks in the above code. The first argument to *Celery* is the name of the module (in our case, *tasks*). We also need to add tasks in the queue wherever necessary, and have workers running to do the work.